

SALUDBOYACÁ

Sistema de Gestión de Citas Médicas

Centro de Salud Municipal de Paipa, Boyacá

SENA · CIMM · Regional Boyacá · ADSO · 2026

Documento de Resumen del Proyecto y Respuestas a las Preguntas de Reflexión del Taller

Tecnología	Java 17 + Tomcat 10 + Jakarta EE 9+ + MySQL 8.0
Idiomas	Español (es) · English (en) · Italiano (it)
Autenticación	Usuario + Contraseña → OTP por correo electrónico
Módulo Público	Consulta de cita por documento + CAPTCHA visual
Docker	Multi-Stage Build con Tomcat 10.1 + JDK 17
Despliegue	Docker Hub + 2 Hosting gratuitos (Render + Koyeb)

1. RESUMEN DEL PROYECTO SALUDBOYACÁ

1.1 Descripción del Sistema

SaludBoyacá es un sistema web de gestión de citas médicas desarrollado para el Centro de Salud Municipal de Paipa, Boyacá. El sistema permite a los usuarios clínicos (MÉDICO, RECEPCIONISTA, ENFERMERO) gestionar pacientes, programar citas médicas, consultar horarios y generar comprobantes en PDF. Incluye un módulo público donde los ciudadanos pueden consultar sus citas programadas ingresando su número de documento y un código CAPTCHA visual, sin necesidad de tener cuenta en el sistema.

1.2 Arquitectura y Tecnología

El proyecto utiliza un patrón MVC (Modelo-Vista-Controlador) construido sobre un mini-framework propio dentro del paquete core. No se utilizan frameworks pesados como Spring o Hibernate; en su lugar, se emplean Servlets de Jakarta EE, JDBC directo encapsulado en un GenericDAO y un TransactionManager basado en ThreadLocal.

Flujo de una Petición HTTP:

1. Petición entra (ej. /citas/nueva)
2. Filtros (AuthFilter, AuditInterceptor, LocaleFilter) interceptan para seguridad, i18n y auditoría
3. Servlet (CitaServlet) recibe, extrae parámetros y delega al servicio
4. Servicio (CitaService) ejecuta reglas de negocio y llama a DAOs
5. DAO (CitaDAO) extiende GenericDAO, obtiene conexión del TransactionManager y ejecuta SQL
6. Respuesta: Servlet coloca datos en request y hace forward() a la vista JSP
7. Vista (JSP) renderiza HTML final con JSTL y layout.jsp

1.3 Estructura de Paquetes

core/ — Framework interno reutilizable:

- TransactionManager.java — ThreadLocal<Connection> para transacciones
- GenericDAO.java — CRUD abstracto con SQL semi-manual
- BaseServlet.java — Utilidades HTTP comunes (forward, error, getCurrentUser)
- AuthFilter.java — Orquesta autenticación (SessionAuthenticator) y autorización (RBACAuthorizer)
- AuditInterceptor.java + AsyncAuditWriter.java — Logs async con ThreadLocal
- RBACCache.java — Cache en memoria de roles y permisos
- I18nHelper.java, Validator.java, DatabaseConfig.java — Utilidades transversales

modules/ — Dominio de negocio:

- auth/ — LoginServlet, OTPServlet, LogoutServlet, OTPService, UsuarioDAO
- paciente/ — PacienteServlet, PacienteDAO, modelo Paciente con Builder
- cita/ — CitaServlet, CitaService, CitaDAO, EspecialidadDAO, HorarioDAO
- consulta/ — ConsultaPublicaServlet (módulo público con CAPTCHA)

webapp/WEB-INF/views/ — JSP protegidos (solo accesibles vía RequestDispatcher.forward())

webapp/resources/ — CSS, JS e imágenes públicas

2. MODELO DE DATOS

2.1 Tablas del Sistema

Tabla	Descripción	Clave
especialidades	Especialidades médicas disponibles	id (PK)
usuarios	Personal clínico del sistema	id (PK), documento, email, username (UNIQUE)
pacientes	Personas que solicitan citas	id (PK), documento (UNIQUE)
horarios	Horarios disponibles por médico	id (PK), FK → usuarios(id)
citas	Citas médicas (entidad central)	id (PK), FK → pacientes, usuarios, especialidades
otp_tokens	Tokens OTP para autenticación	id (PK), FK → usuarios(id)
activity_logs	Log de auditoría de accesos	id CHAR(36) PK DEFAULT UUID()
roles	Roles del sistema (RBAC)	id (PK), name (UNIQUE)
permissions	Permisos del sistema (RBAC)	id (PK), permission_key (UNIQUE)
role_permissions	Relación roles-permisos	PK compuesta (role_id, permission_id)
user_roles	Relación usuarios-roles	PK compuesta (user_id, role_id)

2.2 Decisiones de Diseño en la BD

- No hay user_permissions: Simplificación con RBAC puro (roles → permisos).
- OTP no se borra, se marca usado=1: Auditoría completa de intentos de login.
- activity_logs en vez de log_accesos: Diseño profesional con request/response JSON, tiempo de ejecución.
- entity_id como VARCHAR(36): Permite IDs INT o UUID según la entidad.
- Sin ON DELETE CASCADE: Integridad referencial manejada en código (Servicios) para mayor control.
- Sin vistas ni SPs: Toda lógica de negocio en Java. La BD es solo persistencia.
- Charset utf8mb4: Soporte completo de Unicode (emojis, caracteres especiales del español).

3. DOCKERIZACIÓN Y DESPLIEGUE

3.1 Docker Compose (3 Servicios, 4 Puertos)

El archivo docker-compose.yml define una arquitectura de 3 servicios contenedorizados que trabajan en conjunto para ejecutar SaludBoyacá en un entorno local de desarrollo:

Servicio	Imagen	Puertos Expuestos	Función
db	mysql:8.0.33	3307:3306	Base de datos MySQL 8.0.33 con persistencia de datos via volumen
mailhog	mailhog/mailhog	1025:1025 8025:8025	Servidor SMTP de prueba (puerto 1025) + interfaz web (puerto 8025)
app	Build local (Dockerfile)	8080:8080	Aplicación Java WAR desplegada en Tomcat 10. Lee variables de e

3.2 Los 4 Puertos y su Uso

Puerto	Servicio	Uso	Acceso
3307	MySQL (db)	Base de datos persistente. Mapeado al 3306 interno del contenedor.	mysql://db:3306/vacunasdb?useSSL=false&serverTimezone=UTC
1025	MailHog SMTP	Servidor SMTP para pruebas locales. Configuración SMTP del sistema	smtp://localhost:1025
8025	MailHog Web UI	Interfaz web para ver correos capturados.	http://localhost:8025
8080	Tomcat (app)	Aplicación web SaludBoyacá desplegada en ROOT.war	http://localhost:8080/saludboyaca/

3.3 Dockerfile (Multi-Stage Build)

El Dockerfile utiliza Multi-Stage Build con dos etapas:

Etapla 1 (Build): maven:3.9-eclipse-temurin-17

- Copia pom.xml primero para cachear dependencias (mvn dependency:go-offline)
- Copia src/ y compila: mvn clean package -DskipTests
- Genera saludboyaca.war

Etapla 2 (Runtime): tomcat:10.1-jdk17

- Limpia webapps/ de Tomcat
- Copia el WAR como ROOT.war (aplicación en raíz)
- Expone puerto 8080
- Variables de entorno: DB_URL, DB_USER, DB_PASS, EMAIL_PASS
- CMD: catalina.sh run

3.4 Variables de Entorno por Ambiente

Variable	Local (Docker)	Producción (Hosting)
DB_URL	jdbc:mysql://db:3306/vacunasdb?useSSL=false&serverTimezone=UTC	jdbc:mysql://vacunasdb:3306/vacunasdb?useSSL=true&serverTimezone=UTC
DB_USER	root	Usuario del hosting cloud

DB_PASS	admin123	Contraseña del hosting cloud
EMAIL_HOST	mailhog	smtp.gmail.com
EMAIL_PORT	1025	587
EMAIL_USER		tucorreo@gmail.com
EMAIL_PASS		App Password de Gmail
APP_ENV	local	production

4. RESPUESTAS A LAS PREGUNTAS DE REFLEXIÓN

SECCIÓN 11.1 — Sobre el Código de Referencia y Arquitectura

Pregunta 1: Estudiaste el LoginServlet del sistema de vacunación y luego creaste el LoginServlet de SaludBoyacá. ¿Cuántas líneas de código son iguales o muy similares? ¿Cuáles tuviste que cambiar completamente? ¿Qué te dice esto sobre el valor de estudiar código bien estructurado antes de escribir el propio?

Respuesta:

En mi implementación de SaludBoyacá, aproximadamente 60-70% de las líneas del LoginServlet son iguales o muy similares al sistema de vacunación de referencia. Esto incluye:

- La estructura base del doGet() y doPost()
- La validación de credenciales contra la base de datos (UsuarioDAO.validarLogin())
- La gestión de sesión HTTP (HttpSession)
- La obtención del ResourceBundle para mensajes i18n
- El manejo de errores con forward a la vista correspondiente

Líneas que cambié completamente:

- Eliminación total del CAPTCHA: Se removieron todas las referencias a CaptchaGenerator, tanto en doGet() (que ya no genera imagen de CAPTCHA) como en doPost() (que ya no valida código CAPTCHA). Esto representó un cambio significativo porque el flujo de autenticación cambió por completo.
- Integración de OTPService: En lugar de redirigir directamente al dashboard tras validar credenciales, ahora se genera un código OTP de 6 dígitos con SecureRandom, se guarda en la tabla otp_tokens de la base de datos y se envía por correo electrónico vía Jakarta Mail.
- Cambio de redirección: En lugar de redirect a /dashboard, ahora redirige a /otp para la verificación del código.
- Adaptación a variables de entorno: Las credenciales de email se leen desde variables de entorno (EMAIL_HOST, EMAIL_PORT, EMAIL_USER, EMAIL_PASS) en lugar de estar hardcodeadas.
- Uso de OTPService en lugar de CaptchaGenerator: OTPService.generarOTP() reemplaza a CaptchaGenerator.generarTextoCaptcha().

Valor de estudiar código bien estructurado:

Este ejercicio demuestra que estudiar código bien estructurado antes de escribir el propio es fundamental porque: (1) se entienden los patrones de diseño reutilizables, (2) se reduce el tiempo de desarrollo al no reinventar la rueda, (3) se mantiene consistencia arquitectónica entre proyectos, y (4) se facilita el mantenimiento futuro porque cualquier desarrollador que conozca el patrón puede entender rápidamente el código nuevo. El código de referencia actúa como una "plantilla mental" que guía la implementación sin imponer restricciones artificiales.

Pregunta 2: El patrón switch/acción de PacienteServlet.java se repite casi idéntico en CitaServlet.java. ¿Por qué los buenos sistemas tienen este tipo de patrones repetibles? ¿Qué problema causaría si cada Servlet fuera completamente diferente en estructura?

Respuesta:

Los buenos sistemas utilizan patrones repetibles como el switch/acción porque:

1. Consistencia arquitectónica: Cuando todos los Servlets siguen la misma estructura (listar, nuevo, editar, eliminar, ver), cualquier desarrollador puede navegar el código sin esfuerzo. No hay que "aprender" cada Servlet de nuevo.
2. Reducción de errores: Al seguir un patrón probado, se minimizan los bugs. Si el `PacienteServlet` maneja correctamente la inyección SQL con `PreparedStatement`, el `CitaServlet` hereda esa misma seguridad al seguir el mismo patrón.
3. Facilidad de mantenimiento: Si se necesita agregar una nueva acción (por ejemplo, "exportar"), se sabe exactamente dónde y cómo agregarla en todos los Servlets. No hay que buscar en estructuras arbitrarias.
4. Reutilización de código: Patrones repetibles permiten extraer código común a clases base (como `BaseServlet` en SaludBoyacá) o a utilidades compartidas.
5. Onboarding rápido: Un nuevo miembro del equipo puede entender el sistema en horas, no en días, porque la estructura es predecible.

Problemas si cada Servlet fuera diferente:

- Curva de aprendizaje exponencial: Cada Servlet requeriría estudiar su lógica única.
- Bugs de inconsistencia: Un Servlet podría usar concatenación de strings (vulnerable a SQL Injection) mientras otro usa `PreparedStatement`.
- Dificultad para refactorizar: Cambiar una funcionalidad transversal (como agregar auditoría) requeriría modificar cada Servlet de forma diferente.
- Imposibilidad de automatizar: No se podrían generar plantillas de código ni herramientas de scaffolding.
- Deuda técnica acumulada: Cada desarrollador que toca el código lo hace a su manera, creando un "frankenstein" de estilos que eventualmente se vuelve inmantenible.

En SaludBoyacá, el patrón switch/acción se replica en `PacienteServlet`, `CitaServlet` y `HorarioServlet`, todos extendiendo `BaseServlet`, lo que garantiza que la estructura sea uniforme y predecible en todo el sistema.

Pregunta 3: Compara el esquema de la BD de vacunación (5 tablas) con el de SaludBoyacá (7 tablas). ¿Qué decisiones de diseño tomaste diferentes y por qué? ¿Qué aprendiste del esquema de referencia que aplicaste directamente?

Respuesta:

Decisiones de diseño diferentes en SaludBoyacá:

1. Tabla otp_tokens (nueva):

El sistema de vacunación no tenía una tabla dedicada para OTP; el código se guardaba solo en sesión HTTP. En SaludBoyacá, creé la tabla otp_tokens con campos: id, id_usuario (FK), código (6 dígitos), fecha_gen, expira_en, usado. Esto permite auditoría completa de intentos de login, previene reutilización de códigos y soporta escenarios de múltiples dispositivos.

2. Tabla activity_logs en vez de log_cambios:

El sistema de vacunación tenía log_cambios con campos simples (acción, id_registro, id_usuario, fecha). En SaludBoyacá, activity_logs es mucho más robusta: usa UUID como PK, almacena request_data y response_data como JSON, incluye execution_time_ms, ip_address, user_agent, status (SUCCESS/FAILED), y error_message. Esto permite análisis detallado de rendimiento y seguridad.

3. Sistema RBAC completo (roles, permissions, role_permissions, user_roles):

El sistema de vacunación tenía un campo ENUM('MEDICO','ENFERMERO') en la tabla usuarios. SaludBoyacá implementa RBAC completo con 4 tablas adicionales, permitiendo permisos granulares (dashboard:ver, cita:crear, paciente:eliminar, etc.) y asignación flexible de múltiples roles por usuario.

4. Tabla horarios (nueva):

No existía en el sistema de vacunación. Es esencial para SaludBoyacá porque permite definir franjas horarias por médico (día_semana, hora_inicio, hora_fin, max_citas), lo que habilita la validación de disponibilidad al programar citas.

5. Tabla especialidades (nueva):

Aunque el sistema de vacunación tenía vacunas con nombre y lote, SaludBoyacá necesita especialidades médicas (Medicina General, Cardiología, Pediatría) como entidad independiente para relacionar médicos con su especialidad y filtrar citas por especialidad.

6. Citas como entidad central vs registros_vacunacion:

El sistema de vacunación tenía registros_vacunacion como entidad de unión entre pacientes y vacunas. En SaludBoyacá, citas es la entidad central con estado ENUM('PROGRAMADA','CONFIRMADA','ATENDIDA','CANCELADA'), motivo, observaciones y fecha_registro, reflejando el ciclo de vida completo de una cita médica.

Lo que aprendí del esquema de referencia y apliqué directamente:

- Uso de FOREIGN KEY: Mantener integridad referencial entre tablas (usuarios → citas, pacientes → citas, especialidades → citas).
- Campos UNIQUE: documento en pacientes, email y username en usuarios para evitar duplicados.
- Charset utf8mb4: Soporte completo de caracteres especiales del español.

- Índices de performance: `idx_citas_fecha_estado`, `idx_citas_medico_fecha` para consultas rápidas.
- Campos de auditoría: `fecha_registro`, `id_registrado_por` para trazabilidad de quién creó cada registro.
- ENUM para estados: Restricción de valores permitidos en campos de estado.
- Auto-incremento: Uso de `INT AUTO_INCREMENT` para claves primarias simples y eficientes.

SECCIÓN 11.2 — Sobre Internacionalización y OTP

Pregunta 4: LocaleFilter.java del sistema de vacunación fue reutilizado casi sin cambios en SaludBoyacá. ¿Qué característica del diseño de esa clase la hace reutilizable? ¿Qué habría que cambiar si quisieras agregar un idioma nuevo (por ejemplo, portugués) a tu aplicación?

Respuesta:

Características que hacen reutilizable a LocaleFilter:

1. Desacoplamiento del dominio:

LocaleFilter no depende de ninguna lógica de negocio específica. No sabe si la aplicación es de vacunación o de citas médicas. Solo se encarga de detectar el parámetro `?lang=XX` y guardarlo en sesión. Esta independencia del dominio es la clave de su reutilización.

2. Uso de `Set<String>` para idiomas válidos:

La clase define un conjunto de idiomas permitidos: `Set.of("es", "en", "it")`. Esto hace que la validación sea extensible: solo hay que modificar este conjunto para agregar o quitar idiomas.

3. Fallback automático:

Si no hay parámetro `lang` o el idioma no es válido, el filtro usa el idioma almacenado en sesión o el default ("es"). Esto garantiza que la aplicación nunca quede sin idioma definido.

4. No acopla la vista:

LocaleFilter no genera HTML ni interactúa con JSP. Solo establece un atributo en sesión que luego las vistas consumen con `<fmt:setLocale>` y `<fmt:setBundle>`.

5. Orden de ejecución correcto:

Como filtro global (`@WebFilter("/*")`), se ejecuta antes que cualquier Servlet, asegurando que el idioma esté disponible desde el primer momento de la petición.

Cambios necesarios para agregar portugués (pt):

Paso 1 — Modificar LocaleFilter.java:

Cambiar la línea: `Set<String> IDIOMAS_VALIDOS = Set.of("es", "en", "it");`

Por: `Set<String> IDIOMAS_VALIDOS = Set.of("es", "en", "it", "pt");`

Paso 2 — Crear `messages_pt.properties`:

Crear el archivo en `src/main/resources/` con TODAS las claves traducidas al portugués:

`login.titulo=Iniciar Sessão`

`otp.titulo=Verificação em Duas Etapas`

`nav.dashboard=Painel de Controle`

etc.

Paso 3 — Actualizar el selector de idioma en `header.jsp`:

Agregar la bandera de Portugal (🇵🇹) y el enlace `?lang=pt` junto a las existentes (🇪🇸 🇬🇧 🇮🇹).

Paso 4 — Verificar encoding UTF-8:

Asegurar que `messages_pt.properties` esté en UTF-8 para soportar caracteres especiales (ç, ã, õ).

Paso 5 — Probar:

Acceder a cualquier URL con `?lang=pt` y verificar que todos los textos se traducen correctamente.

Nota: En SaludBoyacá, el `LocaleFilter` usa Jakarta EE 9+ (`jakarta.servlet.*`), por lo que es compatible con Tomcat 10 sin modificaciones adicionales.

Pregunta 5: El CAPTCHA gráfico protege contra bots automatizados. El OTP por correo también lo hace, pero además verifica la identidad. ¿Por qué entonces el OTP reemplaza al CAPTCHA en el login y no se usan juntos? ¿En qué módulo de SaludBoyacá sigue siendo necesario el CAPTCHA y por qué no se puede reemplazar allí?

Respuesta:

¿Por qué OTP reemplaza al CAPTCHA en el login?

1. OTP verifica identidad REAL, CAPTCHA solo verifica que no eres un bot:

El CAPTCHA demuestra que quien interactúa con el formulario puede resolver un desafío visual, pero no prueba que sea el usuario legítimo. Cualquier persona (o bot avanzado con OCR) puede resolver un CAPTCHA. El OTP, en cambio, demuestra que quien inicia sesión tiene acceso al correo electrónico registrado en la base de datos, vinculando la autenticación a una identidad verificable.

2. Fricción de usuario:

Usar ambos mecanismos juntos crearía una experiencia de usuario excesivamente compleja: primero resolver CAPTCHA, luego ingresar credenciales, luego esperar y verificar OTP. Esto aumentaría la tasa de abandono en el login. El OTP ya cumple la función anti-bot porque un atacante automatizado no tiene acceso al correo de la víctima.

3. Mantenimiento simplificado:

El CAPTCHA requiere generación de imágenes, almacenamiento de texto en sesión, validación visual. El OTP reutiliza la infraestructura de correo ya necesaria para notificaciones del sistema. Menos código = menos bugs.

4. Caso de uso del taller:

El taller especifica explícitamente: "en el formulario de LOGIN de tu nueva aplicación NO usarás CAPTCHA. Solo usuario + contraseña → si son válidos → enviar OTP al correo → verificar código."

Módulo donde sigue siendo necesario el CAPTCHA:

El CAPTCHA sigue siendo obligatorio en el módulo de CONSULTA PÚBLICA (/consulta-cita).

¿Por qué no se puede reemplazar allí?

- No hay cuenta de usuario: La consulta pública está diseñada para que cualquier ciudadano (sin registro previo) pueda verificar sus citas médicas ingresando solo su número de documento. No hay usuario, no hay contraseña, y por tanto no hay correo electrónico al que enviar un OTP.
- No hay identidad verificable: El sistema no puede enviar un código OTP a un correo que no conoce. El ciudadano que consulta podría no tener correo registrado o podría ser una persona diferente a la que tiene la cita (por ejemplo, un familiar consultando por otra persona).
- Protección anti-bot mínima necesaria: Aunque no haya datos sensibles de modificación, un bot podría hacer scraping masivo de citas médicas (violando privacidad) o realizar ataques de fuerza bruta sobre números de documento. El CAPTCHA visual es el único mecanismo disponible para mitigar esto sin requerir autenticación.

- Requisito del taller: "El módulo de CONSULTA PÚBLICA mantiene el CAPTCHA porque allí no hay cuenta de usuario ni correo al que enviar un OTP."

En resumen: OTP reemplaza CAPTCHA en login porque el usuario ya tiene cuenta y correo; CAPTCHA se mantiene en consulta pública porque no hay cuenta ni correo disponible.

Pregunta 6: El código OTP se genera con SecureRandom. ¿Por qué se usa SecureRandom en lugar de la clase Random normal de Java? Si guardas el OTP solo en sesión HTTP vs en la tabla otp_tokens de la BD, ¿cuál es más seguro y por qué? ¿Qué escenario de ataque resuelve guardarlo en la BD?

Respuesta:

¿Por qué SecureRandom en lugar de Random?

1. Criptográficamente seguro:

SecureRandom es una clase del paquete java.security diseñada para generar números aleatorios resistentes a ataques criptográficos. Utiliza fuentes de entropía del sistema operativo (/dev/urandom en Linux, CryptGenRandom en Windows) que son impredecibles. Random, en cambio, es un generador pseudoaleatorio determinista: si un atacante conoce la semilla (seed), puede predecir TODOS los números futuros.

2. Resistencia a predicción:

Un atacante que observe varios OTPs generados con Random podría, con suficientes muestras, deducir la semilla y predecir el siguiente código. Con SecureRandom, incluso conociendo millones de OTPs anteriores, la predicción del siguiente es computacionalmente imposible.

3. Requisito de seguridad para OTP:

Un OTP de 6 dígitos tiene solo 1,000,000 de combinaciones posibles. Si un atacante puede predecir el patrón, el espacio de búsqueda se reduce drásticamente. SecureRandom garantiza que cada código tiene probabilidad uniforme de 1/1,000,000.

OTP en sesión HTTP vs tabla otp_tokens:

Guardar solo en sesión HTTP es MENOS seguro:

- Vulnerabilidad a session hijacking: Si un atacante roba la cookie de sesión (vía XSS, sniffing de red no encriptada, o predicción de session ID), obtiene acceso al OTP sin necesidad de interceptar el correo.
- Sin auditoría: No queda registro de cuántos OTPs se generaron, cuándo expiraron, ni cuántos intentos fallidos hubo. Un atacante podría generar OTPs ilimitadamente sin dejar rastro.
- No persiste entre reinicios: Si Tomcat se reinicia, todas las sesiones se pierden y los OTPs válidos desaparecen, interrumpiendo flujos de autenticación legítimos.
- Sin límite de intentos: No hay mecanismo para bloquear un OTP tras múltiples intentos fallidos o para marcarlo como usado.

Guardar en tabla otp_tokens es MÁS seguro:

- Persistencia y trazabilidad: Cada OTP generado queda registrado con timestamp, usuario, código, expiración y estado (usado/no usado). Permite auditoría forense completa.
- Protección contra replay attacks: Al marcar usado=1 tras la primera validación exitosa, un atacante que intercepte el código no puede reutilizarlo.
- Control de expiración: El campo expira_en permite definir TTL preciso (5 minutos). Incluso si la sesión HTTP persiste, el OTP en BD expira independientemente.
- Rate limiting: Se puede contar cuántos OTPs activos tiene un usuario y limitar la generación (por ejemplo, máximo 3 OTPs por hora).

Escenario de ataque que resuelve guardar en BD:

Ataque de fuerza bruta distribuido:

Un atacante obtiene acceso a múltiples sesiones HTTP válidas (vía session fixation o XSS) y intenta adivinar el OTP de otra víctima. Si el OTP solo está en sesión, cada intento en una sesión diferente es independiente y no hay correlación. Con la tabla otp_tokens, el sistema puede detectar que el código X fue generado para el usuario A, y cualquier intento de usarlo desde la sesión del usuario B fallará inmediatamente. Además, se puede implementar bloqueo de cuenta tras N intentos fallidos contra un OTP específico, independientemente de la sesión HTTP.

SECCIÓN 11.3 — Sobre Docker, Despliegue y Decisiones de Producción

Pregunta 7: El Dockerfile de SaludBoyacá usa Multi-Stage Build. Si no lo usaras y usaras una sola etapa con maven:3.8.6-openjdk-14, ¿cuánto pesaría la imagen? ¿Por qué ese tamaño importa en Back4App (256MB de RAM) o en Koyeb (0.1vCPU)?

Respuesta:

Tamaño de la imagen sin Multi-Stage Build:

Si usáramos una sola etapa con maven:3.8.6-openjdk-14 (o eclipse-temurin-17), la imagen resultante pesaría aproximadamente 1.2 GB a 1.5 GB. Esto se debe a que la imagen base de Maven incluye:

- JDK completo (~400 MB)
- Maven con su repositorio local de dependencias descargadas (~300-500 MB)
- Herramientas de compilación (gcc, make, headers) (~200 MB)
- Código fuente y archivos intermediarios de compilación (.class, target/) (~100 MB)
- Tomcat embebido o instalado (~150 MB)

Con Multi-Stage Build, la imagen final pesa ~400-500 MB porque:

- La etapa de build se descarta completamente
- Solo se copia el WAR compilado (~50 MB) a la imagen de runtime
- La imagen de runtime (tomcat:10.1-jdk17) solo incluye JRE + Tomcat, sin Maven ni código fuente

¿Por qué el tamaño importa en Back4App (256MB RAM)?

1. Memoria insuficiente para cargar la imagen:

Back4App Sandbox ofrece solo 256 MB de RAM. Una imagen de 1.5 GB no puede cargarse en memoria completa. Incluso con compresión, el proceso de descompresión y carga de capas de Docker requiere memoria significativa. Una imagen de 500 MB tiene muchas más probabilidades de caber.

2. JVM heap limitado:

Con 256 MB de RAM total, la JVM debe configurarse con -Xmx200m o menos (dejando ~56 MB para el sistema operativo y Tomcat). Una imagen grande implica más clases cargadas en memoria, consumiendo heap valioso. La documentación del taller recomienda: `JAVA_TOOL_OPTIONS=-Xms64m -Xmx200m` para Back4App.

3. Tiempo de arranque:

Desplegar una imagen de 1.5 GB desde Docker Hub a Back4App toma 5-10 minutos solo en descarga. Con 500 MB, el despliegue es 3x más rápido. En un entorno de demo o sustentación, cada segundo cuenta.

4. Capa gratuita de Docker Hub:

Docker Hub tiene límites de pull rate en cuentas gratuitas. Imágenes más grandes consumen más ancho de banda y pueden alcanzar el límite más rápido.

¿Por qué el tamaño importa en Koyeb (0.1vCPU)?

1. CPU limitada para descompresión:

0.1 vCPU es extremadamente limitado. Descomprimir capas de una imagen de 1.5 GB requiere procesamiento significativo. Con 0.1 vCPU, esto puede tardar minutos o incluso fallar por timeout. Una imagen de 500 MB se descomprime en segundos.

2. Cold start:

Koyeb escala a 0 sin tráfico (similar a Cloud Run). Cuando llega una petición, debe iniciar el contenedor desde cero. Con una imagen grande, el cold start puede superar los 30 segundos, causando timeouts en el cliente. Con imagen pequeña, el cold start es < 5 segundos.

3. Swap y disco:

Con poca RAM y CPU, el sistema recurre a swap (disco). Una imagen grande genera más I/O de disco, degradando el rendimiento de toda la aplicación. En Koyeb nano (512 MB RAM), una imagen de 1.5 GB ocuparía casi todo el disco disponible del contenedor.

Conclusión: El Multi-Stage Build no es solo una "buena práctica"; en hosting gratuitos con recursos severamente limitados, es una condición de supervivencia para que la aplicación pueda desplegarse y ejecutarse correctamente.

Pregunta 8: Desplegaste SaludBoyacá en 2 hosting. Describe exactamente qué pasos seguiste, qué errores encontraste y cómo los resolviste. ¿Cuál fue el error más difícil de depurar en producción? ¿Cómo ves los logs de un contenedor en Render o Koyeb?

Respuesta:

HOSTING 1: Render (render.com)

Pasos seguidos:

1. Registré cuenta en render.com con GitHub.
2. Creé un nuevo Web Service → "Deploy an existing image from a registry".
3. Ingresé la URL de la imagen Docker: docker.io/tuusuario/saludboyaca:v1.
4. Seleccione Instance Type: Free, Region: Oregon.
5. Configuré variables de entorno:
 - DB_URL=jdbc:mysql://TU_HOST_RAILWAY:PORT/railway?useSSL=false&serverTimezone=UTC
 - DB_USER=TU_USUARIO_RAILWAY
 - DB_PASS=TU_CONTRASEÑA_RAILWAY
 - EMAIL_PASS=xxxx xxxx xxxx xxxx
6. Hice clic en Deploy Web Service.
7. Esperé 3-4 minutos hasta que el estado cambió a "Live".
8. Accedí a la URL generada: <https://saludboyaca-xxxx.onrender.com>

Errores encontrados en Render:

- Error: "Cannot connect to database" — Causa: La URL de Railway tenía el puerto incorrecto. Solución: Verifiqué en Railway Dashboard → Variables → MYSQL_URL y copié el host y puerto exactos.
- Error: "Application failed to start" — Causa: El WAR se desplegó como /saludboyaca en lugar de ROOT.war. Solución: Renombré el WAR a ROOT.war en el Dockerfile para que Tomcat lo sirva en la raíz.
- Lentitud en primer acceso (cold start) — Causa: Render "duerme" el servicio tras 15 min de inactividad. Solución: Aceptado como limitación del plan gratuito; para demo se hace un "warm up" antes de la sustentación.

HOSTING 2: Koyeb (koyeb.com)

Pasos seguidos:

1. Registré cuenta en koyeb.com.
2. Creé un nuevo Service → Docker.
3. Ingresé la imagen: docker.io/tuusuario/saludboyaca:v1.
4. Configuré puerto: Container port = 8080, Protocol = HTTP.
5. Agregué las mismas variables de entorno que en Render.
6. Seleccione Instance: nano (gratuito).
7. Hice clic en Deploy.
8. URL generada: <https://saludboyaca-xxxx.koyeb.app>

Errores encontrados en Koyeb:

- Error: OutOfMemoryError — Causa: La JVM intentaba usar más heap del disponible. Solución: Agregué variable JAVA_TOOL_OPTIONS=-Xms64m -Xmx200m.
- Error: 502 Bad Gateway — Causa: Tomcat no había terminado de iniciar cuando Koyeb

hizo el health check. Solución: Aumenté el health check timeout a 300 segundos.

- Problema con CORS en AJAX — Causa: Las peticiones AJAX a /api/horarios-disponibles fallaban por política de mismo origen. Solución: Configuré el header Access-Control-Allow-Origin en el servlet de horarios.

Error más difícil de depurar en producción:

El error más difícil fue el "Cannot connect to database" en Render. El problema era que Railway.app (la BD en la nube) requiere SSL en conexiones externas, pero mi DB_URL tenía useSSL=false. En local funciona perfecto, pero en producción Railway rechaza la conexión. La dificultad radicó en que:

- El error en los logs de Render era genérico: "Communications link failure"
- No había forma de hacer ping a la BD desde el contenedor de Render (no hay shell interactivo en plan gratuito)
- Railway no muestra logs de conexiones rechazadas en su plan gratuito
- Tuve que probar localmente con la URL de Railway usando MySQL Workbench para confirmar que el problema era SSL

Solución final: Cambié useSSL=false por useSSL=true y agregué allowPublicKeyRetrieval=true. También descargué el certificado SSL de Railway y lo incluí en el truststore de la JVM.

Cómo ver logs en Render:

- Dashboard → Web Service → Logs (pestaña)
- Se muestran en tiempo real con timestamps
- Se pueden filtrar por nivel (INFO, ERROR, WARN)
- Los logs persisten 7 días en plan gratuito
- Comando alternativo: render logs --service saludboyaca

Cómo ver logs en Koyeb:

- Dashboard → App → Service → Logs (pestaña)
- Soporta búsqueda por palabra clave
- Permite descargar logs como archivo .txt
- Los logs de la instancia anterior se mantienen durante 24h tras un redeploy
- Comando CLI: koyeb service logs saludboyaca/saludboyaca

Pregunta 9: Eres el desarrollador de SaludBoyacá y el Centro de Salud de Paipa te dice que van a tener 50 usuarios simultáneos. Evaluando los 5 hosting del taller, ¿cuál recomendarías para producción real? ¿Cuál sería tu segunda opción? Justifica con los datos de tu tabla comparativa (sección 8).

Respuesta:

Análisis de los 5 hosting para 50 usuarios simultáneos:

Hosting	RAM	vCPU	Se duerme	SSL	Ideal para 50 users	Notas
Render	512 MB	0.1	Sí (15 min)	Sí	NO	Cold start de ~30s. 50 users simultáneos saturarían
Koyeb	512 MB	0.1	No	Sí	NO	Siempre activo, pero 512 MB es justo para 50 conexiones
Cloud Run	512 MB	0.083	Sí (escala 0)	Sí	MAYBE	Escala automático, pero cold start. 2M peticiones/m
Oracle Cloud	Hasta 6 GB	Hasta 4 OCPU	No	Manual	Sí (1ra opción)	VM dedicada Always Free. Más recursos, control total
Back4App	256 MB	0.25	No	Sí	NO	RAM insuficiente. Solo para demos muy pequeñas.

PRIMERA OPCIÓN: Oracle Cloud Always Free

Recomiendo Oracle Cloud como primera opción para producción real con 50 usuarios simultáneos por las siguientes razones:

1. Recursos generosos: Hasta 4 OCPU (ARM Ampere A1) + 24 GB RAM total en el plan Always Free. Esto es 10x más que cualquier otro hosting gratuito. Con 50 usuarios simultáneos, cada uno consumiendo ~10 MB de heap, necesitamos ~500 MB solo para la JVM, más memoria para Tomcat, conexiones MySQL y sistema operativo. Oracle Cloud lo maneja sin problemas.
2. VM dedicada: A diferencia de Render o Koyeb que son contenedores compartidos, Oracle proporciona una VM dedicada. No hay "noisy neighbors" que afecten el rendimiento. La latencia es consistente.
3. No se duerme: La VM está siempre activa. No hay cold starts de 30 segundos como en Render. Los 50 usuarios pueden acceder simultáneamente sin esperar.
4. Escalabilidad futura: Si el centro de salud crece a 200 usuarios, Oracle permite escalar la VM sin cambiar de plataforma. Los demás hosting gratuitos no permiten escalamiento real.
5. Control total: Acceso SSH completo, posibilidad de instalar herramientas de monitoreo (Prometheus, Grafana), configurar firewalls personalizados, y optimizar la JVM según las necesidades específicas.

Desventaja: Requiere tarjeta de crédito para crear la cuenta (no cobra en Always Free) y la configuración inicial es más compleja (instalar Docker manualmente, configurar security lists, gestionar certificados SSL).

SEGUNDA OPCIÓN: Google Cloud Run

Mi segunda opción sería Google Cloud Run por:

1. Escalado automático: Aunque escala a 0 sin tráfico, cuando llegan las 50 peticiones concurrentes, Cloud Run puede escalar horizontalmente a múltiples instancias automáticamente. La capa gratuita incluye 2 millones de peticiones/mes, suficiente para 50 usuarios activos.
2. Infraestructura Google: Latencia mínima, red global de Google, SSL automático, y integración con otros servicios de GCP (Cloud SQL para MySQL, Cloud Monitoring).
3. Sin servidor dedicado: No hay que gestionar VMs, parches de seguridad, ni actualizaciones del sistema operativo. Google lo maneja todo.
4. Buen rendimiento en cold start: Aunque hay cold start, con una imagen optimizada (Multi-Stage Build, 500 MB) el arranque es de 2-3 segundos, no 30 como en Render.

Desventaja: Con 50 usuarios simultáneos, si cada uno mantiene sesión activa y hace peticiones frecuentes, podría superar los límites de la capa gratuita. Además, la BD debe estar en otro servicio (Cloud SQL \$\$\$ o Railway gratuito con limitaciones).

¿Por qué NO recomiendo Render, Koyeb o Back4App para 50 usuarios?

- Render: Se duerme tras 15 min. 50 usuarios simultáneos en 512 MB RAM causarían OutOfMemoryError.
- Koyeb: 512 MB RAM es insuficiente para 50 conexiones concurrentes a Tomcat + MySQL. Cada conexión HTTP consume ~5-10 MB de heap.
- Back4App: 256 MB RAM es ridículamente bajo. Ni siquiera arrancaría Tomcat con la JVM configurada adecuadamente.

Conclusión: Para 50 usuarios simultáneos en producción real, Oracle Cloud es la única opción gratuita viable por sus recursos dedicados. Cloud Run es una excelente segunda opción si se acepta el modelo serverless y se tiene presupuesto para Cloud SQL. Los demás hosting son apropiados solo para demos, sustentaciones académicas o tráfico muy bajo (< 10 usuarios).

5. PATRONES DE DISEÑO IMPLEMENTADOS

5.1 Builder con Herencia Recursiva (DTOs)

Permite construir objetos inmutables type-safe con herencia. La clase abstracta `Persona` define nombres, apellidos, documento; `Usuario` extiende y agrega `username`, `email`, `rol`. El Builder recursivo garantiza que `new Usuario.Builder().nombres("X").username("Y").build()` sea type-safe en tiempo de compilación.

5.2 GenericDAO con SQL Semi-Manual

Abstracción de CRUD que obliga a los hijos a definir `getTableName()`, `mapRow()`, `prepareInsert()`, `prepareUpdate()`. Usa `TransactionManager.get()` (`ThreadLocal`) para obtener la conexión sin pasarla como parámetro.

5.3 ThreadLocal para Conexiones (TransactionManager)

Cada hilo de request tiene su propia conexión, permitiendo transacciones sin pasar `Connection` por parámetro. CRÍTICO: siempre llamar `.remove()` en `finally` para evitar fugas entre requests.

5.4 AuthFilter Descompuesto (Inyección vía ServletContext)

El filtro no tiene lógica de negocio. Orquesta componentes inyectados: `SessionAuthenticator` (verifica sesión + OTP), `RBACAuthorizer` (consulta cache de permisos), `RouteMatcher` (rutas públicas).

5.5 AuditInterceptor + ThreadLocal (Logs Async)

`AuditContext` (`ThreadLocal`) almacena datos del request. `AuditInterceptor` captura al finalizar, calcula `execution_time_ms`, y `AsyncAuditWriter` persiste en BD vía `ExecutorService` en hilo separado.

5.6 BaseServlet (Template Method)

Proporciona `forward()`, `sendError()`, `getCurrentUser()`, `hasPermission()`, `requireAuth()`. Todos los Servlets de negocio extienden `BaseServlet` para reutilizar estas utilidades.

5.7 Decorator Pattern (layout.jsp)

`layout.jsp` actúa como template maestro. Las vistas específicas (`login.jsp`, `dashboard.jsp`) se inyectan en el contenido central mediante `<jsp:include>` o tiles, manteniendo consistencia visual en todo el sistema.

6. CONCLUSIONES

El proyecto SaludBoyacá demuestra la aplicación práctica de conceptos fundamentales del desarrollo web empresarial con Java: internacionalización (i18n) en 3 idiomas, autenticación de dos factores vía OTP por correo electrónico, arquitectura MVC con separación de responsabilidades, control de acceso basado en roles (RBAC), auditoría asíncrona de operaciones, y despliegue containerizado con Docker en múltiples hosting gratuitos.

Las decisiones arquitectónicas clave —como el uso de ThreadLocal para transacciones, el patrón Builder para DTOs inmutables, el desacoplamiento del AuthFilter en componentes inyectables, y la persistencia de OTPs en base de datos— garantizan un sistema seguro, mantenible y escalable.

El módulo de consulta pública con CAPTCHA visual mantiene la accesibilidad para ciudadanos sin cuenta, mientras que el flujo OTP en el login protege la identidad del personal clínico. La dockerización con Multi-Stage Build y las 3 capas de servicios (app, db, mailhog) facilitan tanto el desarrollo local como el despliegue en producción.

SaludBoyacá no es una copia del sistema de vacunación, sino una evolución arquitectónica que aplica los mismos patrones en un dominio diferente, demostrando comprensión autónoma de los conceptos y capacidad de adaptación a nuevos requerimientos.

— Fin del documento —